

Progetto ASD 2026: Algoritmi per Grafi di Autonomous Systems

docente: Roberto Grossi, tutor: Tommaso Dossi

May 2026

1 Descrizione

Il progetto ha l'obiettivo di analizzare la topologia di rete per la trasmissione dei dati su internet a livello di Autonomous Systems (AS), utilizzando statistiche reali derivate dal protocollo BGP (Border Gateway Protocol) per instradare i pacchetti di dati (packet routing).

Le statistiche BGP permettono di osservare i percorsi tra AS utilizzati nel routing di internet. A partire da tali informazioni è possibile costruire un grafo in cui i nodi rappresentano gli Autonomous Systems e gli archi rappresentano connessioni osservate nei cammini BGP.

Il progetto si concentra in particolare sulla definizione di una funzione di costo per gli archi, poiché i grafi AS disponibili pubblicamente sono generalmente non pesati. Non è necessaria la conoscenza del protocollo BGP per svolgere i compiti assegnati nel progetto. L'idea principale consiste nell'assegnare un peso agli archi utilizzando la frequenza con cui essi compaiono nei cammini BGP osservati:

$$w(u, v) = f(u, v) \quad (1)$$

dove $f(u, v)$ rappresenta il numero di volte in cui l'arco (u, v) compare nei cammini BGP osservati. In questo modo gli archi frequentemente utilizzati avranno costo maggiore.

1.1 Dataset

Una possibile sorgente per le statistiche è il dataset CAIDA AS Relationships, disponibile all'indirizzo seguente, dove i file sono testuali e compressi in formato **bzip2**:

<https://publicdata.caida.org/datasets/as-relationships/>

In particolare, ciascun dataset con nome del tipo

https://publicdata.caida.org/datasets/as-relationships/serial-1/*.as-rel.txt.bz2

contiene relazioni tra Autonomous Systems derivate da osservazioni BGP e fornisce una rappresentazione del grafo AS. Una riga del dataset è del tipo

AS1|AS2|relationship

e indica l'esistenza dell'arco non orientato tra **AS1** e **AS2**. Il valore associato alla relazione può essere ignorato (per completezza, è pari a 0 nel caso di collegamenti peer-peer e pari a -1 nel caso provider-customer).

Per ottenere direttamente dump reali dei cammini BGP è possibile utilizzare i dati RIPE (Reti IP Europee), disponibili tramite RouteViews in file del tipo

https://publicdata.caida.org/datasets/as-relationships/serial-1/*.all-paths.bz2

Una riga del dataset è del tipo

```
routeviews/routeviews|5 1239|6113|8063|3464 199.88.21.0/24 i 144.228.240.93
```

e indica che quel cammino BGP attraversa gli AS 1239, 6113, 8063, e 3464. Quindi occorre aumentare di uno la frequenza per gli archi (1239, 6113), (6113, 8063) e (8063, 3464).

1.2 Algoritmi e Analisi sperimentale

Si consideri il grafo pesato e non orientato AS, come descritto sopra. Si noti che potrebbe non essere completamente connesso, in tal caso si consideri la componente connessa più grande. Il progetto richiede la progettazione e l'implementazione dei seguenti algoritmi con apposite strutture di dati.

1. Per un grafo AS, costruire una *struttura di dati* per rispondere alle seguenti *query*: dati due nodi u e v , trovare il **costo** del cammino **minimax ottimo** da u a v , dove il peso di un arco è la sua frequenza $f(u, v)$, e il costo di un cammino è definito come la massima frequenza tra quelle degli archi attraversati dal cammino. Formalmente, dato un cammino di nodi

$$P = (u_0, u_1, \dots, u_k)$$

il suo costo è definito come

$$\text{costo}(P) = \max_{0 \leq i < k} f(u_i, u_{i+1})$$

e si vuole minimizzare tale quantità tra tutti i cammini dove $u_0 = u$ e $u_k = v$. (Il problema è quindi un problema di minimizzazione del massimo peso attraversato, noto anche come problema di cammino minimax.)

2. [OPZIONALE] Dato il grafo AS e due nodi u e v , progettare un algoritmo per **contare** tutti i cammini a costo minimax ottimo nel grafo AS secondo la definizione precedente, ovvero tutti i cammini il cui costo minimax è uguale al valore ottimo. L'analisi sperimentale dovrà riportare:
 - il numero di nodi e archi del grafo;
 - la distribuzione delle frequenze sugli archi;
 - il **costo** ottimo di un cammino minimax;
 - il **numero** di cammini a costo minimax ottimo trovati;
 - i tempi di esecuzione degli algoritmi.

Volendo, si possono contare solo quei cammini a costo minimax ottimo da u a v che sono riportati nel file `*.all-paths.bz2`.

1.3 Nota importante sulla consegna del progetto

La consegna del solo codice in C++ non è sufficiente per superare l'esame. È necessario documentare il **processo** che ha portato alla realizzazione del codice, utilizzando un repository su github.com nel quale siano effettuati commit e push periodici, in modo da conservare gli snapshot della progressione del progetto.

Prima di iniziare la scrittura del codice, occorre inserire nel repository un documento contenente una descrizione sufficientemente dettagliata di come si intende procedere nello sviluppo. Il documento partirà da uno schema iniziale essenziale e sarà successivamente raffinato. Anche per il

documento dovranno essere effettuati vari commit e push, così da mostrare il processo concettuale attraverso cui è evoluto.

Dopo aver predisposto il documento iniziale, si potrà iniziare a scrivere il codice, continuando a effettuare commit e push periodici. È possibile sviluppare in parallelo sia il documento sia il codice, purché il documento rifletta lo stato del codice in quel momento.

La discussione del progetto richiede di fornire al docente l'accesso al repository `github.com`, aggiungendolo come utente `robertogrossi`. Durante la discussione il docente esaminerà i vari commit presenti nel repository e potrà chiedere chiarimenti e motivazioni sulle scelte progettuali e implementative adottate durante lo sviluppo.

È consentito utilizzare strumenti di AI generativa durante la realizzazione del progetto. Tuttavia, è necessario conoscere in modo **approfondito** il contenuto del codice prodotto e saperne **spiegare/modificare** il funzionamento. In caso contrario, il progetto non sarà considerato sufficiente per il superamento dell'esame.

Non sarà considerata valida ai fini della consegna una cronologia di commit nella quale documento e codice compaiano già nella loro forma finale o quasi, poiché ciò non permette di ricostruire il percorso progettuale e implementativo che ha portato alla realizzazione del progetto.

1.4 Esempio

Si considerino i seguenti cammini BGP osservati, dove gli AS sono numerati da 1 a 5:

```
1 2 5
1 2 5
1 3 4 5
1 3 5
2 3 4
3 4 5
```

Le frequenze degli archi e il grafo pesato e non orientato AS risultante sono mostrati in Figura 1.

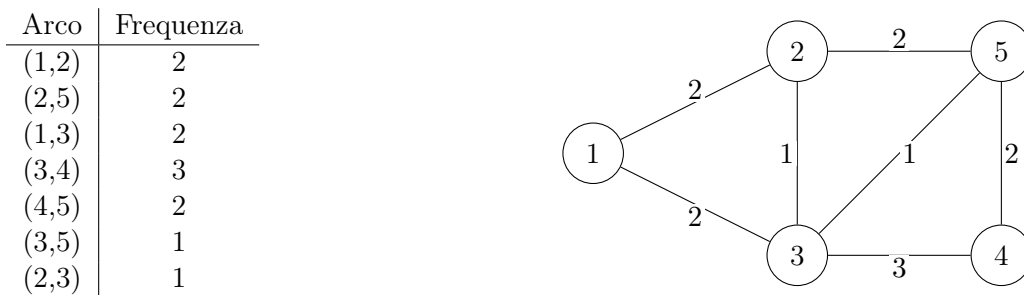


Figura 1: Esempio di grafo AS con frequenze sugli archi.

Si considerino ora i nodi $u = 1$ e $v = 5$. I cammini $1 \rightarrow 3 \rightarrow 5$ e $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$ hanno entrambi costo 2, poiché $\max(2, 1) = 2$ e $\max(2, 1, 1) = 2$ mentre il cammino $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ ha costo 3, dato che $\max(2, 3, 2) = 3$.

Pertanto il valore ottimo minimax tra i nodi 1 e 5 è 2. Un cammino minimax ottimo è $1 \rightarrow 3 \rightarrow 5$, mentre $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ non è ottimo perché attraversa un arco di frequenza 3. Nota: non è necessario usare i cammini minimi di Dijkstra, per costruire la struttura di dati.

Opzionale. Il numero totale di cammini a costo minimax ottimo nel grafo AS da u a v nel grafo è 4. Il numero di cammini minimi da u a v che sono riportati in BGP sono 3 (notare che non può superare il numero precedente).

APPENDICE

A Struttura del documento di progetto

Il documento che accompagna il codice sorgente deve descrivere in modo chiaro e progressivo il processo di progettazione e sviluppo del software. Il documento non deve essere scritto soltanto alla fine del progetto, ma deve evolvere gradualmente insieme al codice, tramite commit e push periodici sul repository `github.com`.

La struttura del documento deve seguire un processo di raffinamento successivo. In una prima fase è sufficiente una descrizione generale dell'architettura del software e degli obiettivi principali. Successivamente il documento dovrà essere raffinato progressivamente, introducendo livelli crescenti di dettaglio fino ad arrivare a una corrispondenza precisa tra documento e codice implementato.

Il documento dovrebbe inizialmente identificare:

- i principali moduli software;
- i compiti assegnati a ciascun modulo;
- le strutture dati fondamentali;
- le principali interazioni tra moduli.

In una fase successiva, ciascun modulo deve essere descritto più dettagliatamente, specificando:

- input e output;
- specifiche funzionali;
- strutture dati utilizzate;
- complessità attesa degli algoritmi;
- eventuali invarianti mantenuti dalle strutture dati;
- dipendenze da altri moduli.

Ogni modulo può essere ulteriormente suddiviso in sotto-moduli. Anche i sotto-moduli devono essere descritti attraverso raffinamenti successivi, fino a raggiungere una descrizione sufficientemente dettagliata da corrispondere direttamente alle componenti implementative presenti nel codice sorgente.

Esempio

Un possibile raffinamento potrebbe partire da un modulo generale per la gestione del grafo AS:

ASGraph

Tale modulo potrebbe successivamente essere raffinato nei seguenti sotto-moduli:

- caricamento dei dataset;
- costruzione del grafo;
- gestione delle frequenze sugli archi;

- ricerca dei cammini minimax;
- conteggio dei cammini minimax ottimi;
- analisi sperimentale.

Per il sotto-modulo

costruzione del grafo

una possibile specifica potrebbe essere la seguente.

- **Input.** Sequenza di cammini BGP letti dai file `*.all-paths.bz2`. Ogni cammino è una sequenza di identificatori numerici di Autonomous Systems.
- **Output.** Grafo non orientato pesato $G = (V, E)$, dove:
 - ogni nodo rappresenta un AS;
 - ogni arco rappresenta una relazione d'adiacenza osservata in un cammino BGP;
 - il peso di un arco rappresenta la frequenza con cui l'arco compare nei cammini osservati.
- **Funzionalità richieste.**
 - Inserimento dinamico dei nodi del grafo.
 - Inserimento degli archi non orientati.
 - Aggiornamento delle frequenze sugli archi.
 - Eliminazione di self-loop (essendo un grafo non orientato).
 - Possibile eliminazione di duplicati consecutivi nei cammini BGP.
- **Strutture dati suggerite.**
 - Tabelle hash per mappare gli identificatori AS in interi consecutivi.
 - Liste di adiacenza implementate come **vector** per rappresentare il grafo.
 - Tabelle hash per memorizzare e aggiornare le frequenze degli archi.
- **Complessità attesa.** La costruzione del grafo dovrebbe richiedere tempo lineare rispetto alla lunghezza totale dei cammini BGP letti in input.

A loro volta, questi sotto-moduli possono essere ulteriormente raffinati in componenti più specifiche, descrivendo classi, funzioni, strutture dati ausiliarie e dettagli implementativi. Questo approccio aiuta a non perdere il filo durante lo sviluppo del codice: occorre chiarirsi il prossimo obiettivo nel documento prima di passare alla scrittura del codice.

Il documento deve quindi rappresentare non soltanto una descrizione statica del software finale, ma soprattutto il processo concettuale che ha guidato la progettazione e l'implementazione del progetto.

Durante la discussione del progetto, il docente potrà esaminare sia il codice sia l'evoluzione progressiva del documento tramite la cronologia dei commit presenti sul repository.